Laporan Tugas Besar 2

Pemanfaatan Algoritma BFS dan DFS dalam Mekanisme Penelusuran CSS pada pohon Document Object Model

IF2211 Strategi Algoritma

Semester II 2025/2026

Disusun oleh Kelompok maaf ya bang aku gak ngerti tubes ini

Nathaniel Christian	13524122
Azri Arzaq Pohan	13524139
Rasyad Satyatama	13524142

Program Studi Teknik Informatika
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung
2026

DAFTAR ISI

DAFTAR ISI **1**
BAB I
DESKRIPSI TUGAS **1**
BAB II
LANDASAN TEORI **2**
2.1 DOM 2
2.2 CSS Selector 2
2.3 HTML Parsing 2
2.4 Algoritma Breadth First Search (BFS) 3
2.5 Algoritma Depth First Search (DFS) 3
BAB III
APLIKASI DFS dan BFS **4**
3.1. Pengaplikasian Breadth-First Search (BFS) 4
3.2 Pengaplikasian Depth-First Search (DFS) + Multithreading 4
BAB IV
IMPLEMENTASI DAN PENGUJIAN **6**
4.1 Algoritma 6
4.2 Testing 12
4.2.1 Expected Input dan Output 12
4.2.2 Invalid Input and/or Output 15
BAB V
KESIMPULAN DAN SARAN **17**
5.1 Kesimpulan 17
5.2 Saran 17
LAMPIRAN **18**
DAFTAR PUSTAKA **20**

BAB I
DESKRIPSI TUGAS

Aplikasi yang dibangun merupakan aplikasi berbasis web dengan backend yang menggunakan bahasa pemrograman Go. Spesifikasi fungsionalitas yang harus dipenuhi adalah sebagai berikut:

- Input data: Aplikasi dapat menerima input berupa URL website atau teks HTML mentah, pilihan algoritma (BFS/DFS), CSS Selector (seperti Tag, Class, ID, atau Combinator), dan jumlah kemunculan hasil yang diinginkan (top n atau semua).
- HTML parsing: Mengonversi hasil scraping menjadi struktur data pohon (DOM Tree).
- Visualisasi dan output:
    a. Menampilkan visualisasi struktur pohon DOM beserta keterangan kedalaman maksimum (max depth).
    b. Memberikan highlight pada jalur penelusuran yang dilalui algoritma untuk menandai elemen yang terpengaruh.
    c. Menampilkan metrik performa berupa waktu pencarian dan jumlah node yang dikunjungi.
    d. Menghasilkan traversal log yang mencatat setiap tahapan penelusuran secara mendetail.
- Fitur tambahan (yang di implementasi):
    a. Implementasi animasi penelusuran pohon secara real-time.
    b. Deployment aplikasi menggunakan Docker dan Microsoft Azure VM.

BAB II
LANDASAN TEORI

2.1 DOM

Document Object Model (DOM) adalah representasi terstruktur dari sebuah dokumen HTML dalam bentuk pohon (*tree*). Struktur ini memungkinkan setiap elemen dalam halaman web untuk diakses, dimodifikasi, dan dimanipulasi secara terprogram. Dalam DOM, setiap elemen HTML direpresentasikan sebagai sebuah node, di mana hubungan antar elemen membentuk hierarki orang tua (*parent*), anak (*child*), dan saudara (*sibling*).

Biasanya, struktur file HTML memiliki elemen root <html> yang terbagi menjadi dua bagian utama, yaitu <head> (metadata) dan <body> (bagian isi utama).

2.2 CSS Selector

CSS Selector adalah mekanisme yang digunakan untuk memilih elemen HTML tertentu pada struktur DOM agar dapat diberikan aturan gaya visual. Selector memungkinkan penentuan target elemen berdasarkan kriteria tertentu:

- Tag selector, yaitu memilih elemen berdasarkan nama tag (misal: p, div)
- Class selector, yaitu memilih elemen dengan atribut class tertentu, ditandai dengan titik (misal: .primary, .box)
- ID selector, yaitu memilih elemen unik dengan id tertentu, biasanya ditandai dengan pagar (misal: #header)
- Universal selector, yaitu memilih semua elemen dalam dokumen, ditandai dengan tanda bintang (*)

Selain itu, terdapat combinator yang merupakan cara untuk menggabungkan beberapa jenis selector dalam penentuan elemen yang akan terpengaruh, dan terdiri dari:

- Child combinator (>), yaitu memilih elemen yang merupakan anak langsung dari elemen tertentu.
- Descendant combinator ("""), yaitu memilih semua elemen yang berada di dalam elemen tertentu, tanpa mempedulikan kedalamannya.
- Adjacent sibling combinator (+), yaitu memilih sebuah elemen yang berada tepat setelah elemen tertentu (*immediately follows*) dan keduanya berbagi parent yang sama.
- General sibling combinator (~), yaitu memilih semua elemen yang berada setelah elemen tertentu (tidak harus tepat di sampingnya), selama elemen-elemen tersebut masih berada di bawah parent yang sama.

2.3 HTML Parsing

HTML Parsing adalah proses pembacaan dokumen HTML mentah untuk diubah menjadi struktur data pohon (DOM Tree). Proses ini melibatkan beberapa tahapan transformasi data:

- Bytes, yaitu data mentah yang diterima dari server.
- Characters, yaitu konversi bytes menjadi karakter teks.
- Tokens, yaitu pengelompokan karakter menjadi suatu unit (Tag) seperti StartTag, EndTag, dan isi konten.
- Nodes, yaitu transformasi tokens menjadi objek-objek node yang memiliki properti tertentu.
- DOM, yaitu penyusunan nodes ke dalam struktur hierarki pohon secara bertahap seiring data diterima.

2.4 Algoritma Breadth First Search (BFS)

Breadth First Search adalah algoritma penelusuran graph yang menjelajahi simpul secara melebar, mulai dari simpul akar lalu mengunjungi semua simpul tetangga pada level yang sama sebelum berpindah ke level berikutnya. Dalam konteks DOM, BFS digunakan untuk menemukan elemen yang sesuai dengan selector dengan memeriksa setiap level kedalaman pohon secara berurutan.

2.5 Algoritma Depth First Search (DFS)

Depth First Search adalah algoritma penelusuran graph yang menjelajahi simpul secara mendalam, yaitu menyusuri satu cabang hingga mencapai node daun sebelum kembali melakukan *backtracking*. DFS lebih efektif untuk menemukan elemen yang terletak jauh di dalam struktur hierarki dokumen.

BAB III
APLIKASI DFS dan BFS

Aplikasi ini menggunakan algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS) untuk melakukan simulasi proses parsing dan pencarian elemen dari sebuah struktur pohon DOM berdasarkan penelusuran CSS Selector. Algoritma tersebut digunakan untuk menjelajahi struktur pohon guna menemukan elemen yang sesuai dengan selector yang diberikan. Berikut adalah penjabaran pengaplikasian masing-masing algoritma berdasarkan implementasi *source code*.

3.1. Pengaplikasian Breadth-First Search (BFS)

Algoritma BFS digunakan untuk pencarian yang menyebar secara hierarki, di mana elemen-elemen yang berada pada kedalaman (*depth*) yang lebih atas di pohon DOM akan dievaluasi terlebih dahulu sebelum turun ke elemen yang lebih bawah.

Pada dasarnya, BFS seharusnya bersifat First-In First-Out (FIFO). Namun, karena penelusuran DOM dapat memiliki selektor bertingkat, priority queue diinisialisasi dalam bentuk dynamic array yang disortir ulang setiap kali di iterasi menggunakan fungsi *sort.SliceStable()*.

Disini, *priority* digunakan untuk menempatkan kandidat elemen berdasarkan kedalaman (*depth*) node yang dituju (*level-by-level processing*). Hal ini menjamin bahwa elemen yang paling dekat dengan struktur root akan selalu diuji kemiripannya dengan selector terlebih dahulu sebelum mengevaluasi sub-elemen anaknya.

Saat algoritma mengevaluasi sebuah elemen kandidat yang cocok dengan langkah *selector* saat ini dan memiliki penelusuran selanjutnya, algoritma akan mencatat elemen-elemen kerabat atau anaknya (*RelatedNodes*). Elemen tersebut lalu dimasukkan kembali ke queue untuk dievaluasi pada *stepIndex+1* tanpa merusak urutan kedalaman sisa pohon utama.

3.2 Pengaplikasian Depth-First Search (DFS) + Multithreading

Algoritma DFS digunakan untuk menelusuri elemen menyusur kedalaman secara rekursif hingga mencapai simpul daun (*leaf*) sebelum beralih mengeksplorasi node saudaranya. Arsitektur DFS ini dioptimalkan dengan mengimplementasikan fitur multithreading sehingga proses traversal dibagi ke beberapa thread/worker dan beberapa node dapat diproses secara bersamaan. Mekanisme pengaplikasiannya:

- Goroutine per cabang. Di dalam fungsi utama traverse, setiap iterasi dari child nodes dipecah dengan membangkitkan Goroutine baru. Ini menciptakan thread worker tersendiri bagi setiap cabang, sehingga terjadinya eksplorasi bersamaan untuk simpul-simpul yang berdampingan di seluruh struktur cabang.
- Karena pemanggilan rekursif bercabang secara masif, aplikasi menyinkronkan thread-thread yang menyebar menggunakan *sync.WaitGroup* melalui pemanggilan *wg.Add(1)* dan *wg.Done()*. Ini mencegah eksekusi program utama selesai sebelum seluruh proses pelacakan pada thread turunan selesai.

- Race prevention dilakukan dengan operasi yang membutuhkan perubahan data kolektif dilindungi dengan mekanisme penguncian (lock) tunggal menggunakan sync.Mutex. Penguncian ini diterapkan saat penambahan node yang ditemukan ke dalam hasil (*results.NodeIDs*) atau saat pencatatan tahapan penelusuran pada *log.UpsertLogEntry*.
- Adanya batas pencarian jumlah hasil (Top n kemunculan), sehingga parameter batas tersebut dikontrol menggunakan variabel stop *atomic.Bool dari package sync/atomic. Ketika target jumlah node telah terpenuhi pada suatu thread, program akan langsung menghentikan operasi thread lain di awal eksekusi fungsinya.

BAB IV
IMPLEMENTASI DAN PENGUJIAN

4.1 Algoritma

backend/services/search/BFS.go

```
package search

import (
    "backend/models"
    "sort"
)

type QueueItem struct {
    Node      *models.DOMNode
    StepIndex int
    Order     int
}

func SearchElementBFS(root *models.DOMNode, selector
*models.Selector, amount int) (*models.SearchResult,
*models.SearchLog) {
    if root == nil || selector == nil {
        return nil, nil
    }

    results := models.SearchResult{
        NodeIDs: []int{},
        Results: make(map[int]models.SelectorResult),
    }

    log := models.SearchLog{
        Selector:   *selector,
        SearchType: "BFS",
        Entries:    []models.SearchLogEntry{},
    }

    queue := []QueueItem{}
    enqueueOrder := 0
    enqueue := func(node *models.DOMNode, stepIndex int) {
        queue = append(queue, QueueItem{Node: node, StepIndex:
stepIndex, Order: enqueueOrder})
        enqueueOrder++
    }

    enqueue(root, 0)

    if amount <= 0 {
        amount = int(^uint(0) >> 1)
    }
```

```
    for len(queue) > 0 && len(results.NodeIDs) < amount {
        sort.SliceStable(queue, func(i, j int) bool {
            leftItem := queue[i]
            rightItem := queue[j]

            l := int(^uint(0) >> 1)
            r := int(^uint(0) >> 1)

            if leftItem.Node != nil {
                l = leftItem.Node.Depth
            }
            if rightItem.Node != nil {
                r = rightItem.Node.Depth
            }

            if l != r {
                return l < r
            }

            return leftItem.Order < rightItem.Order
        })

        currentItem := queue[0]
        queue = queue[1:]

        currentNode := currentItem.Node
        currentStepIndex := currentItem.StepIndex

        if currentNode == nil || currentStepIndex < 0 ||
currentStepIndex >= len(selector.Steps) {
            continue
        }

        logEntry := models.SearchLogEntry{
            NodeID:        currentNode.NodeID,
            Depth:         currentNode.Depth,
            CandidateNode:
selector.Steps[currentStepIndex].Compound.Matches(currentNode),
            SelectedNode:
selector.Steps[currentStepIndex].Compound.Matches(currentNode) &&
currentStepIndex == len(selector.Steps)-1,
        }

        log.UpsertLogEntry(logEntry)

        step := selector.Steps[currentStepIndex]

        for _, child := range currentNode.Children {
            enqueue(child, currentStepIndex)
        }
```

```
        if step.Compound.Matches(currentNode) {
            if currentStepIndex == len(selector.Steps)-1 {
                path := make([]int, 0, currentNode.Depth+1)
                for current := currentNode; current != nil; current
= current.Parent {
                    path = append([]int{current.NodeID}, path...)
                }

                if _, exists := results.Results[currentNode.NodeID];
!exists {
                    results.Results[currentNode.NodeID] =
models.SelectorResult{
                        Node: currentNode,
                        Path: path,
                    }

                    results.NodeIDs = append(results.NodeIDs,
currentNode.NodeID)
                }
            } else {
                nextCombinator :=
selector.Steps[currentStepIndex+1].Combinator
                relatedNodes :=
currentNode.GetRelatedNodes(nextCombinator)

                if nextCombinator == models.CombinatorDescendant ||
nextCombinator == models.CombinatorNone {
                    sort.SliceStable(relatedNodes, func(i, j int)
bool {
                        if relatedNodes[i].Depth !=
relatedNodes[j].Depth {
                            return relatedNodes[i].Depth <
relatedNodes[j].Depth
                        }
                        return relatedNodes[i].NodeID <
relatedNodes[j].NodeID
                    })
                }

                for _, relatedNode := range relatedNodes {
                    if len(results.NodeIDs) >= amount {
                        break
                    }

                    enqueue(relatedNode, currentStepIndex+1)
                }
            }
        }
    }
```

```
    return &results, &log
}
```

backend/services/search/DFS.go

```
package search

import (
    "backend/models"
    "sync"
    "sync/atomic"
)

func searchDFS(node *models.DOMNode, selector *models.Selector,
stepIndex int, amount int, results *models.SearchResult, log
*models.SearchLog, mu *sync.Mutex, stop *atomic.Bool) {
    if len(selector.Steps) == 0 || stepIndex < 0 || stepIndex >=
len(selector.Steps) {
        return
    }
    if stop.Load() {
        return
    }

    if stepIndex == 0 {
        traverse(node, func(node *models.DOMNode) bool {
            if stop.Load() {
                return false
            }

            matchElement(node, selector, 0, amount, results, log,
mu, stop)
            return !stop.Load()
        }, stop)
        return
    }

    matchElement(node, selector, stepIndex, amount, results, log,
mu, stop)
}

func SearchElementDFS(root *models.DOMNode, selector
*models.Selector, amount int) (*models.SearchResult,
*models.SearchLog) {
    if root == nil || selector == nil {
        return nil, nil
    }

    if amount <= 0 {
        amount = int(^uint(0) >> 1)
```

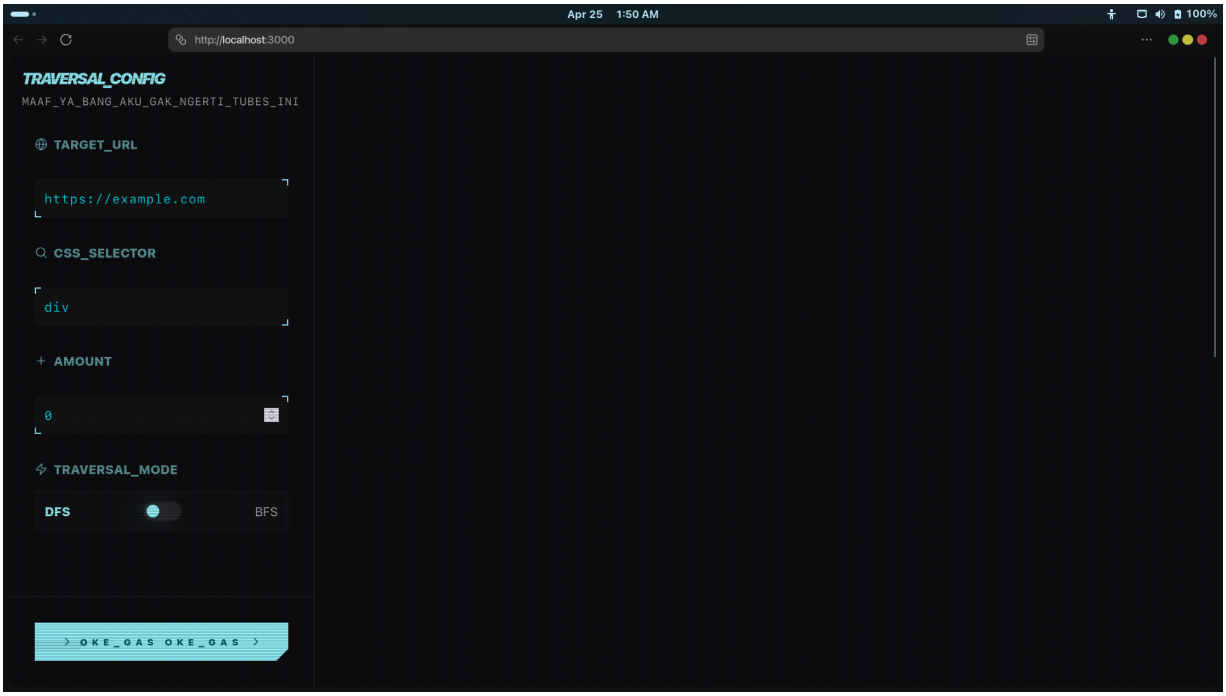
```
    }

    results := models.SearchResult{
        NodeIDs: []int{},
        Results: make(map[int]models.SelectorResult),
    }

    log := models.SearchLog{
        Selector:   *selector,
        SearchType: "DFS",
        Entries:    []models.SearchLogEntry{},
    }

    var mu sync.Mutex
    var stop atomic.Bool

    searchDFS(root, selector, 0, amount, &results, &log, &mu, &stop)
    return &results, &log
}

func matchElement(node *models.DOMNode, selector *models.Selector,
stepIndex int, amount int, results *models.SearchResult, log
*models.SearchLog, mu *sync.Mutex, stop *atomic.Bool) {
    if node == nil || selector == nil || results == nil {
        return
    }
    if stepIndex < 0 || stepIndex >= len(selector.Steps) {
        return
    }
    if stop.Load() {
        return
    }

    isCandidate := selector.Steps[stepIndex].Compound.Matches(node)

    mu.Lock()
    log.UpsertLogEntry(models.SearchLogEntry{
        NodeID:        node.NodeID,
        Depth:         node.Depth,
        CandidateNode: isCandidate,
        SelectedNode:  isCandidate && stepIndex ==
len(selector.Steps)-1,
    })
    mu.Unlock()

    step := selector.Steps[stepIndex]
    if !step.Compound.Matches(node) {
        return
    }

    if stepIndex == len(selector.Steps)-1 {
```

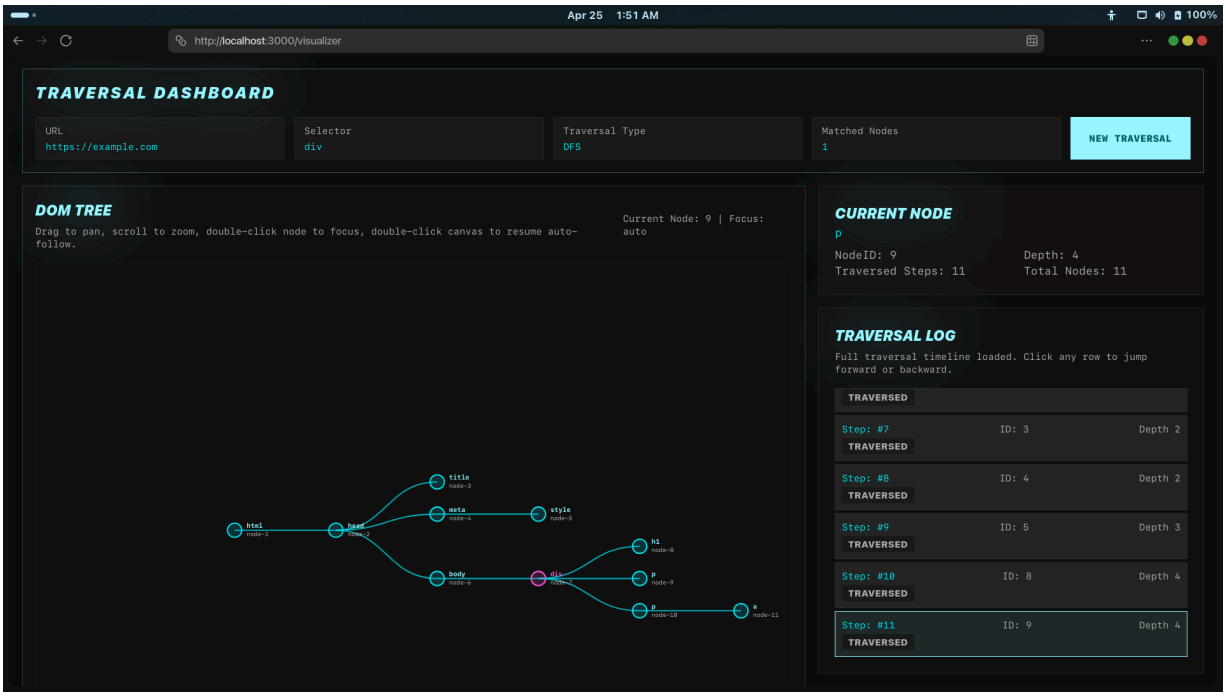
```
        path := make([]int, 0, node.Depth+1)
        for current := node; current != nil; current =
current.Parent {
            path = append([]int{current.NodeID}, path...)
        }

        mu.Lock()
        if stop.Load() {
            mu.Unlock()
            return
        }
        if _, exists := (*results).Results[node.NodeID]; !exists {
            (*results).Results[node.NodeID] = models.SelectorResult{
                Node: node,
                Path: path,
            }

            (*results).NodeIDs = append((*results).NodeIDs,
node.NodeID)

            if len((*results).NodeIDs) >= amount {
                stop.Store(true)
            }
        }
        mu.Unlock()

        return
    }

    nextStep := selector.Steps[stepIndex+1]
    for _, candidate := range
node.GetRelatedNodes(nextStep.Combinator) {
        if stop.Load() {
            return
        }

        matchElement(candidate, selector, stepIndex+1, amount,
results, log, mu, stop)
    }
}

func traverse(node *models.DOMNode, visit func(*models.DOMNode)
bool, stop *atomic.Bool) {
    if node == nil || visit == nil {
        return
    }

    if stop.Load() {
        return
    }
```

```
    if !visit(node) {
        return
    }

    var wg sync.WaitGroup
    for _, child := range node.Children {
        wg.Add(1)
        go func(c *models.DOMNode) {
            defer wg.Done()
            traverse(c, visit, stop)
        }(child)
    }
    wg.Wait()
}
```

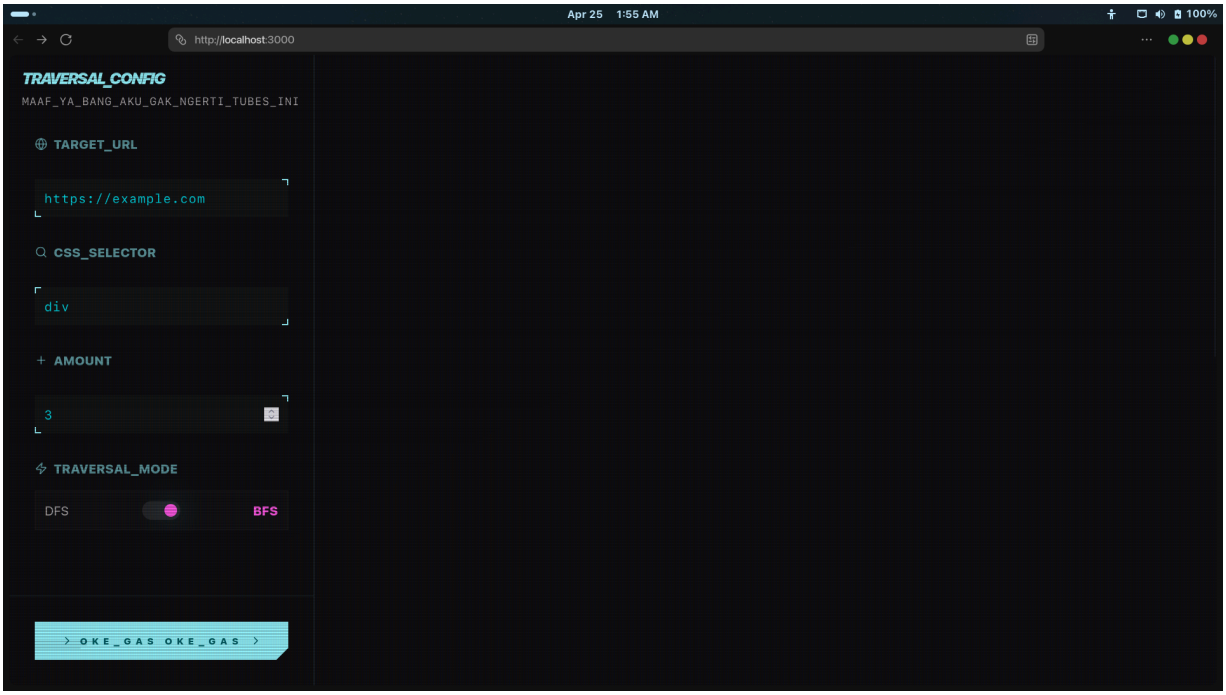
4.2 Testing

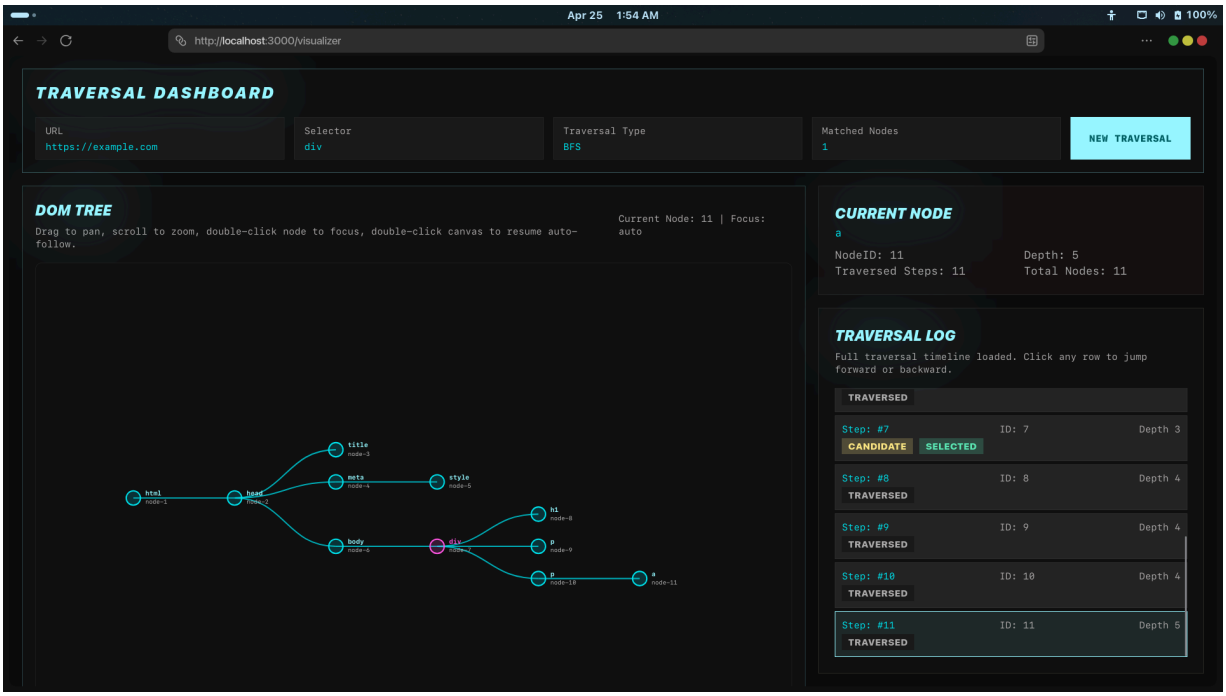
4.2.1 Expected Input dan Output



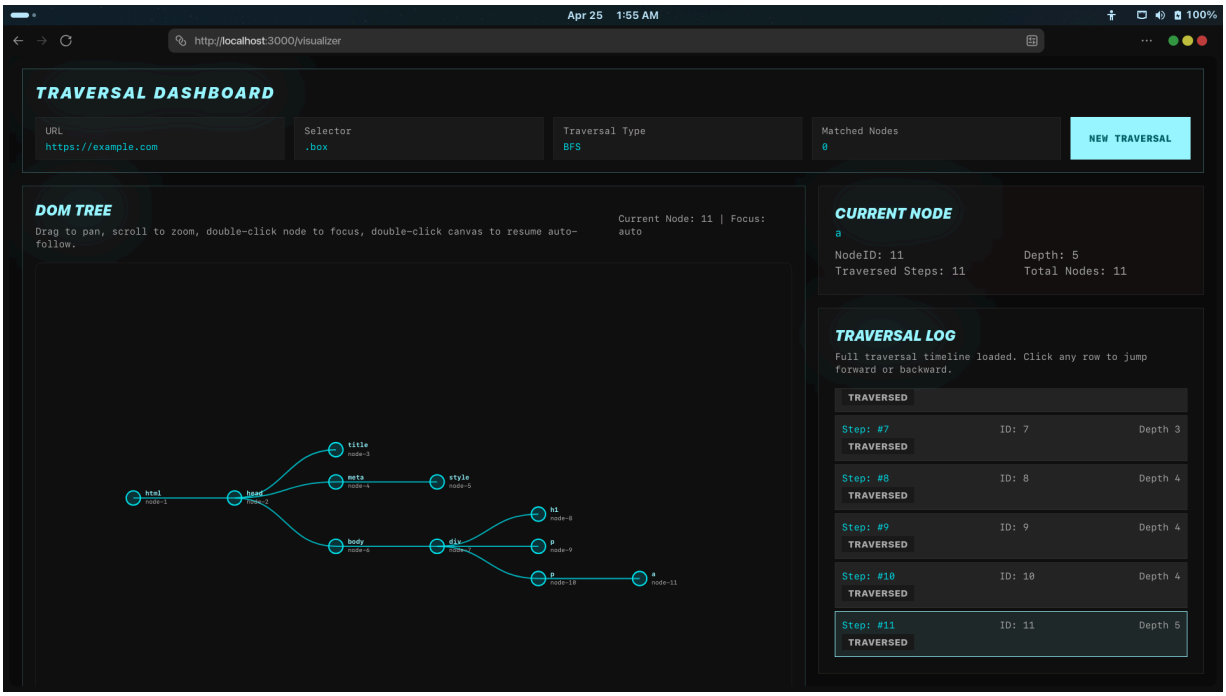


Gambar atas adalah input, menghasilkan gambar bawah/output. Transversal https://example.com, selector div amount 0, DFS.

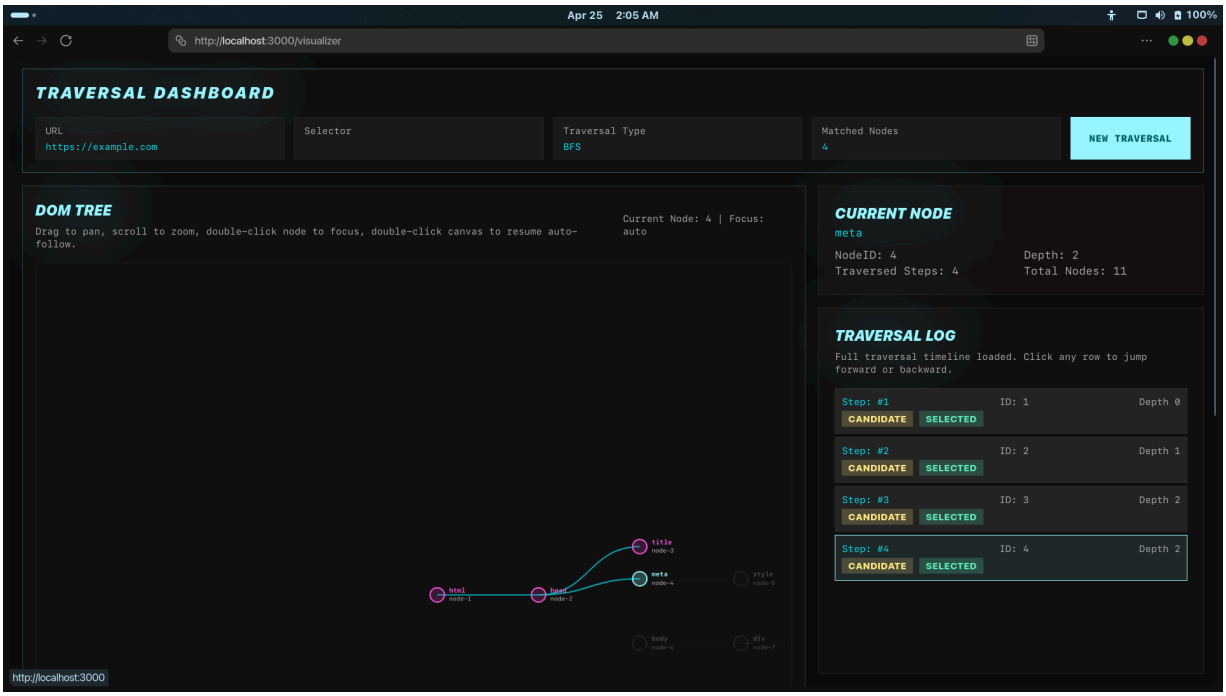




Gambar atas adalah input, menghasilkan gambar bawah/output. Transversal https://example.com, selector div amount 3, BFS

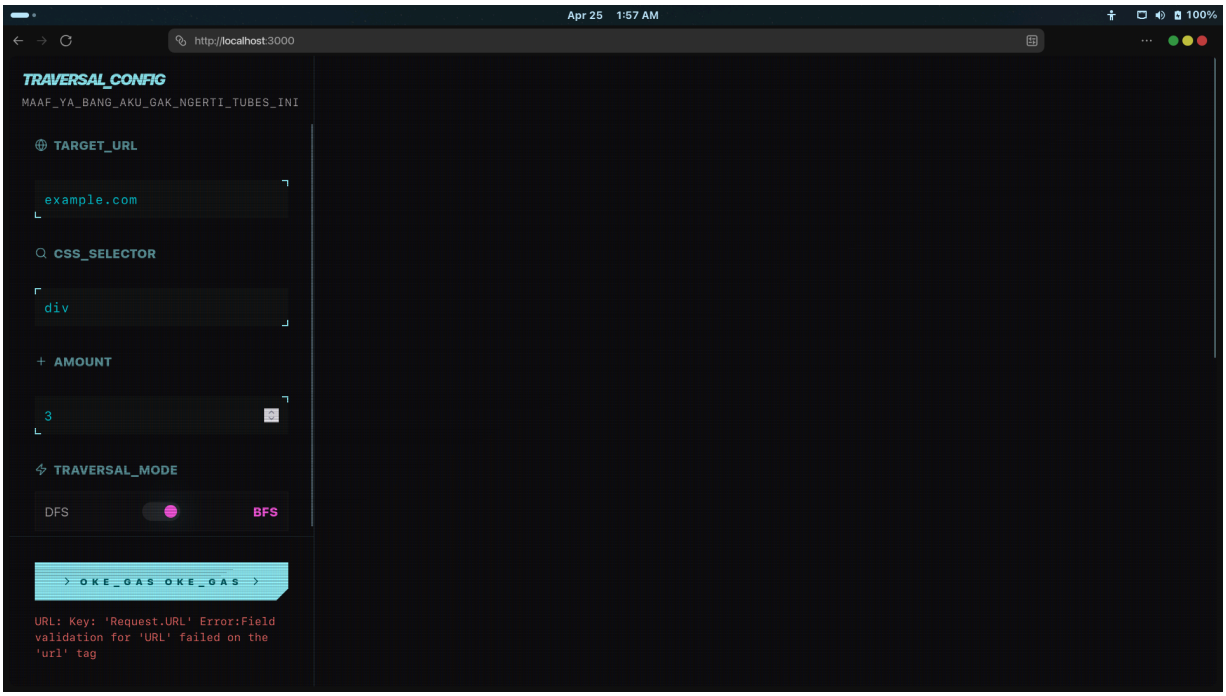


Gambar atas adalah output. Transversal https://example.com, selector .box amount 3, BFS

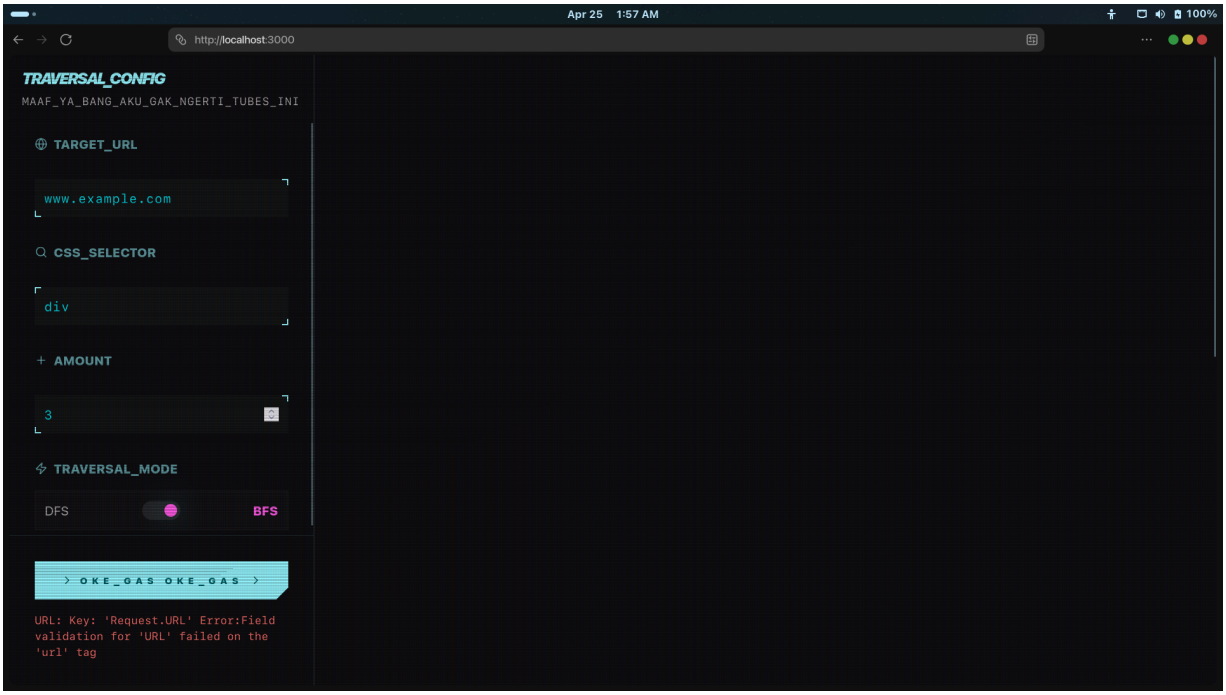


Gambar atas adalah/output. Transversal https://example.com, selector *, amount 4, BFS

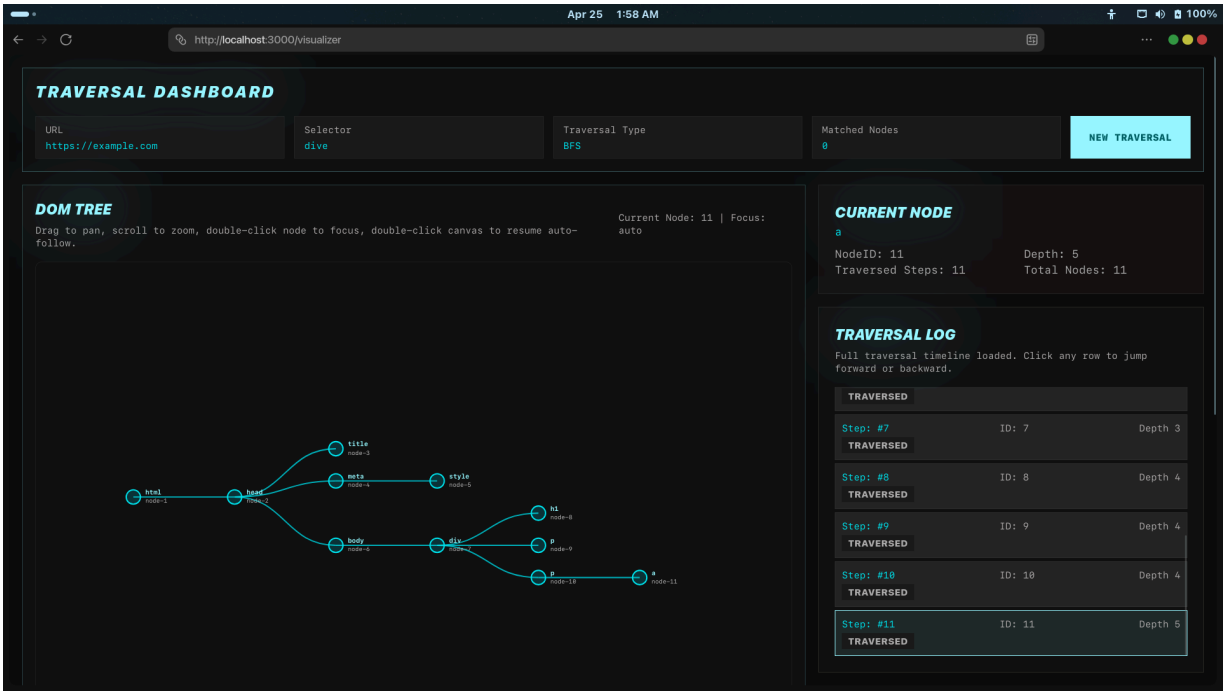
4.2.2 Invalid Input dan/atau Output



Kesalahan di target_url, tidak mempunyai http(s):// di depan url.



Kesalahan di target_url, tidak mempunyai http(s):// di depan url.



Kesalahan penulisan di selector; “dive” harusnya “div”.

BAB V
KESIMPULAN DAN SARAN

5.1 Kesimpulan

Berdasarkan hasil implementasi yang telah dilakukan pada aplikasi penelusuran pohon DOM, dapat ditarik kesimpulan sebagai berikut:

- Algoritma Breadth First Search (BFS) dan Depth First Search (DFS) berhasil diimplementasikan untuk melakukan pencarian elemen pada DOM tree berdasarkan masukan CSS selector yang diberikan.
- BFS efektif menemukan elemen yang terletak pada tingkat kedalaman yang dangkal karena melakukan penelusuran secara melebar level demi level.
- Implementasi DFS dengan teknik multithreading (menggunakan goroutine) memungkinkan proses eksplorasi berbagai cabang pohon dilakukan secara paralel, sehingga dapat meningkatkan efisiensi waktu pencarian pada struktur dokumen HTML yang besar.

5.2 Saran

Nathan	Jaga kesehatan, jangan di opname pas lagi banyak tubes nanti jadi beban di tubes :D Not me tho, 笑死我了
Azri	Makasih atas tumpangannya :3 (Rasyad & Nathan).
Rasyad	Lain kali topiknya disesuaikan ya :)

LAMPIRAN

Github	https://github.com/PikaProgram/Tubes2_maaf-ya-bang-aku-gak-ngerti-tubes-ini/releases/tag/v1.0
Deployment	https://wipe-curse-skewer.ngrok-free.dev/

Pengerjaan:

No	Poin	Ya	Tidak
1	Aplikasi berhasil di kompilasi tanpa kesalahan	✓	
2	Aplikasi berhasil dijalankan	✓	
3	Aplikasi dapat menerima input URL web, pilihan algoritma, CSS selector, dan jumlah hasil	✓	
4	Aplikasi dapat melakukan scraping terhadap web pada input	✓	
5	Aplikasi dapat menampilkan visualisasi pohon DOM	✓	
6	Aplikasi dapat menelusuri pohon DOM dan menampilkan hasil penelusuran	✓	
,7	Aplikasi dapat menandai jalur tempuh oleh algoritma	✓	
8	Aplikasi dapat menyimpan jalur yang ditempuh algoritma dalam traversal log	✓	
9	[Bonus] Membuat video		✓
10	[Bonus] Deploy aplikasi	✓	
11	[Bonus] Implementasi animasi pada penelusuran pohon	✓	
12	[Bonus] Implementasi multithreading	✓	
13	[Bonus] Implementasi LCA Binary Lifting		✓

Pembagian Tugas:

Nathan	Laporan, Testing, Docker, Multithreading
Azri	Backend (scraper, parser)
Rasyad	Frontend (web, animasi), Backend (web, API)

Tugas ini disusun sepenuhnya tanpa bantuan kecerdasan buatan (*Generative AI*), melainkan hasil pemikiran dan analisis mandiri.

(Nathaniel Christian)

(Azri Arzaq Pohan)

(Rasyad Satyatma)

DAFTAR PUSTAKA

MDN contributors (2026) *HTML elements reference*. MDN Web Docs. Available at: https://developer.mozilla.org/en-US/docs/Web/HTML/Reference/Elements (Accessed: 13 April 2026).

MDN contributors (2026) *Void element*. MDN Web Docs. Available at: https://developer.mozilla.org/en-US/docs/Glossary/Void_element (Accessed: 14 April 2026).

MDN contributors (2026) *CSS selectors*. MDN Web Docs. Available at: https://developer.mozilla.org/en-US/docs/Web/CSS/Guides/Selectors (Accessed: 14 April 2026).

MDN contributors (2025) *DOMParser*. MDN Web Docs. Available at: https://developer.mozilla.org/en-US/docs/Web/API/DOMParser (Accessed: 15 April 2026).

https://go.dev/doc/. (Accessed: 16 April 2026).